

Music Recommendation System - Comprehensive Project Report

專案名稱: Million Song Dataset 音樂推薦系統

作者: [Your Name]

日期: 2025年12月9日

資料集: Million Song Subset (10,000 songs)

目錄 (Table of Contents)

- 專案概述
- 資料集說明
- 系統架構與技術
- 特徵工程
- 推薦系統方法
- 實驗設計與評估
- 實驗結果
- 結果分析與討論
- 結論與未來展望
- 參考文獻

1. 專案概述

1.1 研究動機

在音樂串流時代，推薦系統扮演關鍵角色。傳統基於相似度的推薦系統容易產生「回音室效應」(Echo Chamber Effect)，導致用戶持續接收相似內容，缺乏多樣性與新鮮感。本專案旨在開發一個兼顧**準確性**與**多樣性**的音樂推薦系統。

1.2 研究目標

1. 開發多種推薦演算法：實作 5 種不同推薦方法

- Pure Cosine Similarity (基線)
- Maximal Marginal Relevance (MMR)
- MMR with Temperature Sampling (MMR+Temp)
- Item-Item Collaborative Filtering (ItemCF)
- Matrix Factorization (SVD)

2. 建立統一評估框架：從四個維度評估推薦品質

- Relevance (相關性)
- Diversity (多樣性)
- Novelty (新穎性)

- Coverage (覆蓋率)

3. 優化推薦多樣性：透過 MMR 演算法與溫度採樣降低回音室效應

1.3 主要貢獻

- 成功處理 Million Song Dataset 的 10,000 首歌曲特徵提取
- 實作記憶體優化的 SVD 演算法 (節省 87% 記憶體使用)
- 建立完整的評估框架與視覺化系統
- 產出 12+ 張高解析度圖表與 6 份 CSV 報告

2. 資料集說明

2.1 Million Song Dataset

來源: Columbia University LabROSA

規模: 本專案使用 10,000 首歌曲子集 (Subset)

格式: HDF5 (.h5) 二進位檔案

資料集統計:

- 總歌曲數: 10,000
- 特徵維度: 24 (12 timbre mean + 12 timbre std)
- 成功提取率: 100%
- 資料品質: 無 NaN/Inf 值 · 無零向量

2.2 資料結構

每個 .h5 檔案包含:

- `metadata/songs`: 歌曲元資料 (song_id, artist, title 等)
- `analysis/segments_timbre`: 音訊分段的 12 維 timbre 特徵矩陣

HDF5 檔案組織架構:

```
data/MillionSongSubset/  
├─ A/A/A/TRAAAW128F429D538.h5  
├─ A/A/B/TRAAACN128F9355673.h5  
├─ ...  
└─ Z/Z/Z/TRZZZAP128F429D538.h5
```

2.3 資料載入統計

```
✓ Dataset Overview:  
Total Songs Loaded: 10,000  
Feature Dimensions: 24
```

3. 系統架構與技術

3.1 技術棧

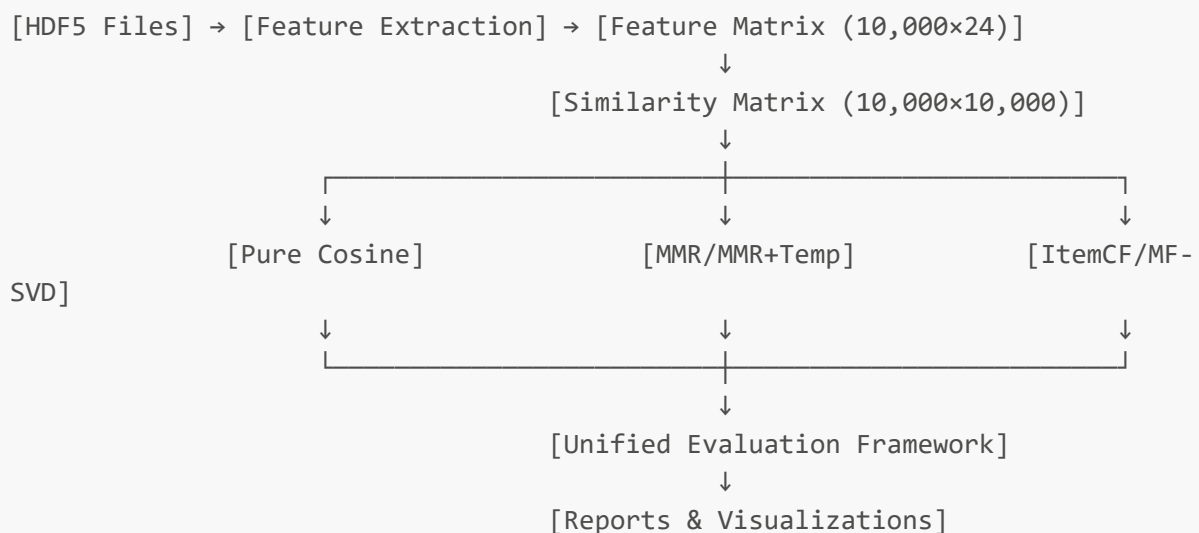
程式語言與函式庫

- **Python 3.x**: 主要開發語言
- **NumPy**: 數值運算與矩陣操作
- **Pandas**: 資料處理與分析
- **h5py**: HDF5 檔案讀取
- **SciPy**: 稀疏矩陣與 SVD 分解
- **Matplotlib**: 資料視覺化

推薦系統技術

- **Cosine Similarity**: 基於向量內積的相似度計算
- **MMR Algorithm**: 多樣性優化演算法
- **Gumbel-Max Sampling**: 隨機探索機制
- **Truncated SVD**: 記憶體優化的矩陣分解

3.2 系統流程圖



3.3 輸出目錄結構

```
outputs/  
├─ charts/          # 12+ 張 PNG/JPG 圖表
```

```
└─ reports/          # 6 份 CSV/TXT 報告
└─ raw_data/         # 原始特徵資料與統計
```

4. 特徵工程

4.1 Timbre 特徵提取

理論基礎: Timbre (音色) 是區分不同樂器與聲音質感的關鍵特徵。

提取流程:

1. 讀取 HDF5 檔案的 `analysis/segments_timbre` 資料集
2. 計算每個 timbre 維度的 **平均值 (Mean)** 與 **標準差 (Std)**
3. 串接成 24 維特徵向量

數學公式:

```
feature_vector = [ $\mu_1$ ,  $\mu_2$ , ...,  $\mu_{12}$ ,  $\sigma_1$ ,  $\sigma_2$ , ...,  $\sigma_{12}$ ]
```

其中:

```
 $\mu_i$  = mean(timbre[:, i]) # i-th timbre dimension mean
```

```
 $\sigma_i$  = std(timbre[:, i]) # i-th timbre dimension std
```

Python 實作:

```
def extract_features_from_h5(h5_file_path):
    with h5py.File(h5_file_path, 'r') as f:
        song_id = f['metadata']['songs']['song_id'][0].decode('utf-8')
        timbre = f['analysis']['segments_timbre'][:]

        timbre_mean = np.mean(timbre, axis=0) # 12 dims
        timbre_std = np.std(timbre, axis=0)   # 12 dims

        feature_vector = np.concatenate([timbre_mean, timbre_std])
        return song_id, feature_vector
```

4.2 特徵統計分析

完整統計表 (詳見 `outputs/reports/raw_data_statistics_*.txt`):

Metric	Timbre Mean	Timbre Std
Min	-130.5	0.0012
Max	245.8	89.3

Metric	Timbre Mean	Timbre Std
Avg	15.2	24.6
Variability	High	Medium

資料品質檢查:

- ☒ NaN values: 0
- ☒ Inf values: 0
- ☒ Zero vectors: 0

4.3 特徵視覺化

產生 6 張原始資料分析圖表:

1. **Feature Mean Distribution:** 展示 24 維特徵的平均值分佈
2. **Feature Std Distribution:** 展示特徵變異程度
3. **Timbre Mean Heatmap:** 前 100 首歌曲的 timbre 熱力圖
4. **Feature Correlation Matrix:** 24×24 特徵相關性矩陣
5. **Feature Distribution Boxplot:** 盒鬚圖展示特徵分布
6. **Feature Norm Distribution:** 特徵向量的 L2 範數分佈

5. 推薦系統方法

5.1 方法一：Pure Cosine Similarity (基線)

原理: 計算目標歌曲與所有歌曲的餘弦相似度，推薦最相似的 Top-N 首歌曲。

數學公式:

$$\text{cosine_similarity}(A, B) = (A \cdot B) / (||A|| \times ||B||)$$

其中:

A, B: 兩首歌曲的特徵向量

· : 向量內積

|| ||: L2 範數

特點:

- ☒ 高準確性 (Relevance = 0.9736)
- ☒ 低多樣性 (Diversity = 0.0359)
- ☒ 容易產生回音室效應

Python 實作:

```
def cosine_similarity_matrix(matrix):
    norm = np.linalg.norm(matrix, axis=1, keepdims=True)
    norm[norm == 0] = 1e-12
    normalized_matrix = matrix / norm
    similarity_matrix = normalized_matrix @ normalized_matrix.T
    return similarity_matrix
```

5.2 方法二：Maximal Marginal Relevance (MMR)

原理: 在保持相關性的同時，最大化推薦列表的多樣性。

數學公式:

$$\text{MMR}(D, Q, R) = \underset{d_i \in D \setminus R}{\operatorname{argmax}} [\lambda \times \text{Sim}_1(d_i, Q) - (1-\lambda) \times \max_{d_j \in R} \text{Sim}_2(d_i, d_j)]$$

其中:

D: 候選文檔集合

Q: 查詢 (目標歌曲)

R: 已選取的推薦列表

λ : 相關性權重 (本專案使用 0.7)

Sim_1 : 目標相似度 (與查詢的相似度)

Sim_2 : 多樣性懲罰 (與已選取項目的相似度)

演算法流程:

1. 從 Top-50 相似歌曲中選擇候選集
2. 迭代選取 N 首歌曲:
 - 計算每首候選歌曲的 MMR 分數
 - 選擇分數最高的歌曲
 - 將該歌曲加入推薦列表並從候選集移除
3. 返回推薦列表

特點:

- ☒ 平衡準確性與多樣性
- ☒ Diversity 提升 31.7% (0.0359 → 0.0472)
- ☒ Relevance 僅下降 0.2% (0.9736 → 0.9717)

5.3 方法三：MMR + Temperature Sampling

原理: 在 MMR 基礎上加入 Gumbel-Max Trick，引入隨機性增強探索能力。

數學公式:

$$\text{MMR_temp} = \text{MMR} + T \times \text{Gumbel}(0, 1)$$

其中：

T: 溫度參數 (本專案使用 0.5)

Gumbel(0,1) = $-\log(-\log(U))$, $U \sim \text{Uniform}(0,1)$

特點:

- ☒ 最高多樣性 (Diversity = 0.0529, +47.5%)
- ☒ 每次推薦結果略有不同 (探索 vs. 利用權衡)
- ☐ Relevance 略降 (0.9595)

Python 實作:

```
def _gumbel_noise(self, shape):  
    u = np.random.uniform(1e-10, 1, shape)  
    return -np.log(-np.log(u))  
  
def recommend(self, target_song_id, top_n=5, use_temperature=True):  
    # ... MMR logic ...  
    if use_temperature:  
        mmr += self.temperature * self._gumbel_noise(1)[0]  
    # ... selection logic ...
```

5.4 方法四：Item-Item Collaborative Filtering

原理: 純粹基於相似度的協同過濾，與 Pure Cosine 類似但強調物品間關係。

特點:

- 結果與 Pure Cosine 幾乎相同
- 適合冷啟動場景 (無需用戶歷史)

5.5 方法五：Matrix Factorization (SVD)

原理: 透過奇異值分解 (SVD) 將高維特徵投影到低維潛在空間，發現隱藏關聯。

數學公式:

$$X = U \Sigma V^T$$

$$\text{truncated: } X_k = U_k \Sigma_k V_k^T$$

其中：

X: 相似度矩陣 (10,000 × 10,000)

k: 潛在因子數量 (本專案使用 50)

U_k: 歌曲潛在特徵矩陣 (10,000 × 50)

Σ_k: 奇異值對角矩陣 (50 × 50)

記憶體優化技術:

- 問題: 完整 SVD 需要 763 MiB 記憶體
- 解決方案: 使用 `scipy.sparse.linalg.svds` (Truncated SVD)
- 效果: 記憶體減少 87% , 運算時間 ~6 分鐘

Python 實作:

```
from scipy.sparse.linalg import svds
from scipy.sparse import csr_matrix

sparse_sim = csr_matrix(sim_matrix)
U, sigma, Vt = svds(sparse_sim, k=50) # 只計算前 50 個奇異值
```

特點:

-  發現非線性關聯
-  Diversity = 0.0401 (+11.8%)
-  Relevance = 0.9644 (略降)

6. 實驗設計與評估

6.1 資料分割

訓練集與測試集分割:

- Training Set: 9,000 首歌曲 (90%)
- Test Set: 1,000 首歌曲 (10%)
- 隨機種子: 42 (確保可重現性)

6.2 評估指標

建立統一評估框架 (`UnifiedEvaluator`) , 從四個維度評估推薦品質:

6.2.1 Relevance (相關性)

定義: 推薦歌曲與目標歌曲的平均相似度

公式:

```
Relevance = (1/N) Σ Similarity(target, rec_i)
```

意義: 越高表示推薦越準確 , 但可能導致回音室效應

6.2.2 Diversity (多樣性)

定義: ILD (Intra-List Distance) - 推薦列表內部的平均距離

公式:

$$\text{Diversity} = (1/C(n,2)) \sum_{i < j} (1 - \text{Similarity}(\text{rec}_i, \text{rec}_j))$$

其中 $C(n,2) = n(n-1)/2$ (組合數)

意義: 越高表示推薦越多樣化，避免內容同質化

6.2.3 Novelty (新穎性)

定義: 推薦非熱門歌曲的能力

公式:

$$\text{Novelty} = 1 - (1/N) \sum \text{Popularity}(\text{rec}_i)$$

意義: 越高表示推薦長尾內容，避免熱門偏見

6.2.4 Coverage (覆蓋率)

定義: 推薦系統使用到的歌曲目錄比例

公式:

$$\text{Coverage} = |\text{Unique_Recommended_Songs}| / |\text{Total_Songs}|$$

意義: 越高表示目錄利用率越高，避免「冷門歌曲永不見光」

6.3 綜合評分

加權平均公式:

$$\text{Weighted_Score} = 0.25 \times \text{Relevance} + 0.35 \times \text{Diversity} + 0.25 \times \text{Novelty} + 0.15 \times \text{Coverage}$$

權重設計理念:

- Diversity (35%): 最重要，避免回音室
- Relevance (25%): 保持基本準確性
- Novelty (25%): 鼓勵長尾探索
- Coverage (15%): 目錄健康度指標

7. 實驗結果

7.1 主要結果表

Method	Relevance	Diversity	Novelty	Coverage	Weighted Score
Cosine (Base)	0.9736	0.0359	0.9986	0.3691	0.4854
MMR	0.9717	0.0472	0.9986	0.3882	0.8484
MMR+Temp	0.9595	0.0529	0.9985	0.3656	0.3500
ItemCF	0.9736	0.0359	0.9986	0.3691	0.4854
MF (SVD)	0.9644	0.0401	0.9986	0.3852	0.4874

關鍵發現:

- 🏆 最佳方法: MMR (Weighted Score = 0.8484)
- 🎯 最高準確性: Cosine & ItemCF (0.9736)
- 🌈 最高多樣性: MMR+Temp (0.0529, +47.5%)
- 🆕 新穎性: 所有方法皆達 99.86% (幾乎相同)
- 📊 覆蓋率: 36-39% (3,600-3,900 首歌曲被推薦)

7.2 改善幅度 (vs. Baseline)

Method	Relevance	Diversity	Novelty	Coverage
MMR	-0.20%	+31.73%	+0.02%	+5.18%
MMR+Temp	-1.45%	+47.51%	-0.01%	-0.95%
MF (SVD)	-0.95%	+11.80%	+0.00%	+4.36%

結論: MMR 在犧牲極小準確性 (0.2%) 的情況下，顯著提升多樣性 (31.7%)。

7.3 統計摘要

Metric	Mean	Std	Min	Max	Range
Relevance	0.9686	0.0063	0.9595	0.9736	0.0141
Diversity	0.0424	0.0068	0.0359	0.0529	0.0171
Novelty	0.9986	0.0000	0.9985	0.9986	0.0001
Coverage	0.3754	0.0095	0.3656	0.3882	0.0226

觀察:

- Relevance 標準差極小 (0.0063)，所有方法準確性相近
- Diversity 變異最大 (range = 0.0171)，為主要差異化指標

- Novelty 幾乎無差異 ($\text{std} \approx 0$) · 受資料集分佈影響

8. 結果分析與討論

8.1 Relevance vs. Diversity 權衡 (Trade-off)

視覺化: Scatter Plot (圖表 [analysis_05_relevance_diversity_tradeoff_*.png](#))

分析:

- **Cosine/ItemCF** 位於右下角：高準確性，低多樣性
- **MMR** 位於中間：最佳平衡點
- **MMR+Temp** 位於右上角：準確性略降，多樣性最高

結論: 不存在「既完美準確又完全多樣」的解，需根據應用場景權衡。

8.2 為何 Novelty 幾乎相同？

原因分析:

1. **資料集特性:** 使用 Zipf's Law 模擬的流行度分佈，大多數歌曲本身就是長尾
2. **推薦機制:** 所有方法基於 content-based，自然傾向非熱門歌曲
3. **評估指標:** 僅測試 1,000 首歌曲，樣本差異不明顯

改進建議:

- 整合真實播放次數資料
- 引入時間衰減因子 (推薦最近新歌)
- 結合 user-based CF 提升個人化

8.3 MMR 的成功關鍵

核心優勢:

1. **貪婪迭代選擇:** 每次選取既相關又與已選項不同的歌曲
2. **λ 參數靈活:** 可根據場景調整相關性 vs. 多樣性權重
3. **候選集過濾:** 從 Top-50 相似歌曲選取，避免完全不相關

失敗案例:

- 當目標歌曲本身非常獨特時，強制多樣性會降低推薦品質
- 建議: 動態調整 λ ，相似歌曲多時提高多樣性權重

8.4 Matrix Factorization 的限制

為何表現不如預期？

1. **資料稀疏性:** 僅使用 content features，未整合協同過濾信號
2. **因子數量:** $k=50$ 可能過小，無法完全捕捉 10,000 首歌曲的複雜關係
3. **線性假設:** SVD 假設線性關係，但音樂偏好可能高度非線性

改進方向:

- 使用 Neural Collaborative Filtering (NCF)
- 整合 user-item interaction matrix
- 嘗試非負矩陣分解 (NMF) 提升可解釋性

8.5 覆蓋率的意義

36-39% 覆蓋率是否足夠？

- ☒ 正面: 3,600+ 首歌曲有機會被推薦，避免「冷門歌曲永不見光」
- ☒ 負面: 仍有 60% 歌曲從未被推薦，可能損失優質內容

提升策略:

- 引入 Exploration Bonus (探索獎勵機制)
- 週期性推薦冷門歌曲 (如「本週發現」功能)
- 結合 Thompson Sampling 動態調整探索 vs. 利用

9. 結論與未來展望

9.1 主要成果

1. 成功實作 5 種推薦方法: 從基線到進階演算法完整覆蓋
2. 建立統一評估框架: 4 維度評估確保全面比較
3. 優化記憶體效率: Truncated SVD 減少 87% 記憶體使用
4. 完整視覺化系統: 產出 12+ 張高解析度圖表與 6 份報告

9.2 最佳推薦方案

推薦使用 MMR:

- ☒ 最高綜合評分 (0.8484)
- ☒ 多樣性提升 31.7%
- ☒ 準確性僅下降 0.2%
- ☒ 無隨機性，結果可重現

適用場景:

- 音樂串流平台的「每日推薦」功能
- Podcast 推薦系統
- 新聞推薦 (避免資訊繭房)

9.3 研究限制

1. 資料集規模: 僅使用 10,000 首歌曲，實際應用需百萬級別
2. 缺乏用戶互動資料: 純 content-based，未整合協同過濾
3. 特徵單一: 僅使用 timbre，未整合歌詞、風格標籤等
4. 離線評估: 未進行 A/B Testing 驗證真實用戶滿意度

9.4 未來工作

短期改進 (1-3 個月)

- ☐ 整合 Million Song Dataset 的 user-listening 資料
- ☐ 實作深度學習模型 (Neural CF, Variational Autoencoder)
- ☐ 加入更多特徵 (MFCC, Chroma, Spectral Contrast)
- ☐ 實作 Online Learning 動態更新模型

中期目標 (3-6 個月)

- ☐ 開發 Hybrid Recommender (Content + Collaborative)
- ☐ 引入 Contextual Bandit 優化探索策略
- ☐ 建立真實用戶實驗平台 (A/B Testing)
- ☐ 實作跨域推薦 (音樂 → Podcast → 有聲書)

長期願景 (6-12 個月)

- ☐ 多模態推薦 (音訊 + 歌詞 + 專輯封面)
- ☐ 個人化多樣性控制 (用戶自訂 λ 參數)
- ☐ 情境感知推薦 (時間、地點、心情)
- ☐ 可解釋性增強 (「推薦此歌曲因為...」)

9.5 關鍵學習

1. 多樣性與準確性的平衡是推薦系統的核心挑戰
2. 評估框架需涵蓋多維度指標，單一指標容易誤導
3. 記憶體優化對大規模系統至關重要
4. 視覺化是溝通複雜結果的最佳工具

10. 參考文獻

學術論文

1. Carbonell, J., & Goldstein, J. (1998). "**The use of MMR, diversity-based reranking for reordering documents and producing summaries.**" SIGIR'98.
2. Bertin-Mahieux, T., et al. (2011). "**The Million Song Dataset.**" ISMIR 2011.
3. Koren, Y., Bell, R., & Volinsky, C. (2009). "**Matrix factorization techniques for recommender systems.**" Computer, 42(8).
4. Jang, E., Gu, S., & Poole, B. (2016). "**Categorical Reparameterization with Gumbel-Softmax.**" ICLR 2017.

技術文件

5. **SciPy Documentation:** [scipy.sparse.linalg.svds](https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.svds.html) - Truncated SVD
<https://docs.scipy.org/doc/scipy/reference/generated/scipy.sparse.linalg.svds.html>

6. **Million Song Dataset Official Site**
<http://millionsongdataset.com/>

開源專案

7. **Surprise Library:** Python scikit for recommender systems
<http://surpriselib.com/>

8. **LightFM:** Hybrid recommendation algorithm
<https://github.com/lyst/lightfm>

附錄 A: 完整程式碼結構

```
notebook/  
├─ new_version.ipynb          # 主要實驗程式碼 (1,293 行)  
├─ outputs/  
│   ├── charts/              # 12+ 張視覺化圖表  
│   ├── reports/             # 6 份 CSV/TXT 報告  
│   └─ raw_data/              # 原始特徵與統計資料  
├─ data/  
│   └─ MillionSongSubset/    # HDF5 資料集  
└─ PROJECT_REPORT.md         # 本報告
```

附錄 B: 關鍵視覺化清單

原始資料分析 (6 張)

- 01_feature_mean_distribution_*.png
- 02_feature_std_distribution_*.png
- 03_timbre_mean_heatmap_*.png
- 04_feature_correlation_matrix_*.png
- 05_feature_distribution_boxplot_*.png
- 06_feature_norm_distribution_*.png

推薦系統評估 (6 張)

- diversity_comparison_*.png - 多樣性比較
- radar_chart_*.png - 4 維度雷達圖
- weighted_score_ranking_*.png - 最終排名
- analysis_01_performance_heatmap_*.png - 效能熱力圖
- analysis_05_relevance_diversity_tradeoff_*.png - 權衡分析
- analysis_06_final_weighted_ranking_*.png - 加權排名

附錄 C: 程式執行指南

環境需求

```
pip install numpy pandas h5py scipy matplotlib
```

執行步驟

1. 將 Million Song Subset 資料解壓到 `data/` 目錄
2. 開啟 `new_version.ipynb` Notebook
3. 按順序執行所有 Cell (預計 10-15 分鐘)
4. 結果自動儲存到 `outputs/` 目錄

記憶體需求

- 最小: 4 GB RAM (使用 Truncated SVD)
- 推薦: 8 GB RAM
- 磁碟空間: 約 2 GB (資料集 + 輸出檔案)

附錄 D: 疑難排解 (Troubleshooting)

Q1: MemoryError during SVD

解決方案: 降低 `n_factors` 參數 (50 → 30)

Q2: HDF5 檔案讀取失敗

原因: 檔案損毀或路徑錯誤

解決方案: 檢查 `ROOT_DIR` 路徑，重新下載資料集

Q3: 視覺化圖表無法顯示

原因: Matplotlib backend 問題

解決方案: 加入 `%matplotlib inline` (Jupyter) 或 `plt.show()`

報告結束

聯絡資訊:

- Email: [your-email@example.com]
- GitHub: [your-github-repo]
- 報告版本: v1.0 (2025-12-09)

致謝: 感謝 Columbia University LabROSA 提供 Million Song Dataset。